Experiment 8

Student Name: Lakshay Aggarwal
UID: 25MCI10047
Branch: MCA (AI/ML)
Section/Group: 25MAM-1(A)
Semester: 2-SEM
Date of Performance: 31/03/2026
Subject Name: Technical Training - 1
Subject Code: 25CAP-652

1. Aim of the Session:

To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

2. Software Requirements:

- PostgreSQL
- pgAdmin
- Windows Operating System

3. Objectives of the Session:

Apply stored procedure concepts for database operation.

4. Theory:

A **Stored Procedure** is a **precompiled set of SQL statements** that is stored inside the database and can be executed whenever required. It is used to perform one or more database operations such as:

1. **INSERT**
2. **UPDATE**
3. **DELETE**
4. **RETRIEVE / SELECT**

Instead of writing the same SQL query again and again, we can create a **stored procedure once** and call it multiple times whenever needed.

Features of Stored Procedure

- Stored inside the database
- Can accept **parameters**
- Can perform **CRUD operations**
- Can return values using **OUT** or **INOUT** parameters

- Can include **conditional statements** like IF, CASE
- Can handle **exceptions / errors**

5. Practical / Experiment Steps:

Step 0: Creating tables and inserting records

Code:

```
CREATE TABLE Employees (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   manager_id INT,
   department VARCHAR(50),
   salary INT
);

INSERT INTO Employees VALUES
(1, 'Rajesh', NULL, 'Management', 130000),
(2, 'Meena', 1, 'Sales', 75000),
(3, 'Suresh', 1, 'Finance', 85000),
(4, 'Anita', 2, 'Sales', 55000),
(5, 'Deepak', 2, 'Sales', 58000),
(6, 'Kavita', 3, 'Finance', 60000),
(7, 'Nitin', 3, 'Finance', 62000),
(8, 'Pallavi', 1, 'HR', 72000),
(9, 'Rohit', 1, 'IT', 90000),
(10, 'Snehal', 9, 'IT', 65000);

SELECT * FROM Employees;
```

Output:

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Rajesh	[null]	Management	130000
2	2	Meena	1	Sales	75000
3	3	Suresh	1	Finance	85000
4	4	Anita	2	Sales	55000
5	5	Deepak	2	Sales	58000
6	6	Kavita	3	Finance	60000
7	7	Nitin	3	Finance	62000
8	8	Pallavi	1	HR	72000
9	9	Rohit	1	IT	90000
10	10	Snehal	9	IT	65000

Step 1: Insertion of Record using Stored Procedure.

Code:

```
CREATE OR REPLACE PROCEDURE add_employee(
   p_id INT,
   p_name VARCHAR,
   p_manager INT,
   p_dept VARCHAR,
   p_salary INT
)
LANGUAGE plpgsql
AS $$
BEGIN
   INSERT INTO Employees VALUES (p_id, p_name, p_manager, p_dept, p_salary);
END;
$$;

CALL add_employee(11, 'Varun', 9, 'IT', 68000);
```

Output:

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Rajesh	[null]	Management	130000
2	2	Meena	1	Sales	75000
3	3	Suresh	1	Finance	85000
4	4	Anita	2	Sales	55000
5	5	Deepak	2	Sales	58000
6	6	Kavita	3	Finance	60000
7	7	Nitin	3	Finance	62000
8	8	Pallavi	1	HR	72000
9	9	Rohit	1	IT	90000
10	10	Snehal	9	IT	65000
11	11	Varun	9	IT	68000

Step 2. Updating the salary of employee using stored procedure

Code:

```
CREATE OR REPLACE PROCEDURE update_salary_procc(
   IN p_emp_id INT,
   INOUT p_salary NUMERIC(20,3),
   OUT status VARCHAR(20)
)
AS
$$
DECLARE
   curr_sal NUMERIC(20,3);

BEGIN
```

```
    SELECT salary + p_salary
    INTO curr_sal
    FROM employees
    WHERE emp_id = p_emp_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'EMPLOYEE NOT FOUND';
    END IF;

    UPDATE employees
    SET salary = curr_sal
    WHERE emp_id = p_emp_id;

    p_salary := curr_sal;
    status := 'SUCCESS';

EXCEPTION
    WHEN OTHERS THEN
        IF SQLERRM LIKE '%EMPLOYEE NOT FOUND%' THEN
            status := 'EMPLOYEE NOT FOUND';
        ELSE
            status := 'FAILED';
        END IF;
END;
$$
LANGUAGE plpgsql;

CALL update_salary_procc(4, 5000, NULL);
```

Output:

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Rajesh	[null]	Management	130000
2	2	Meena	1	Sales	75000
3	3	Suresh	1	Finance	85000
4	5	Deepak	2	Sales	58000
5	6	Kavita	3	Finance	60000
6	7	Nitin	3	Finance	62000
7	8	Pallavi	1	HR	72000
8	9	Rohit	1	IT	90000
9	10	Snehal	9	IT	65000
10	11	Varun	9	IT	68000
11	4	Anita	2	Sales	60000

Step 3: Deleting the record of employee using stored procedure

Code:

```
CREATE OR REPLACE PROCEDURE delete_employee(
    p_id INT
)
LANGUAGE plpgsql
```

```
AS $$
BEGIN
   DELETE FROM Employees
   WHERE emp_id = p_id;
END;
$$;

CALL delete_employee(5);
```

Output:

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Rajesh	[null]	Management	130000
2	2	Meena	1	Sales	75000
3	3	Suresh	1	Finance	85000
4	6	Kavita	3	Finance	60000
5	7	Nitin	3	Finance	62000
6	8	Pallavi	1	HR	72000
7	9	Rohit	1	IT	90000
8	10	Snehal	9	IT	65000
9	11	Varun	9	IT	68000
10	4	Anita	2	Sales	60000

Step4: Retrieving the information of employee

CODE:

```
CREATE OR REPLACE PROCEDURE get_employee_by_id(
 IN p_emp_id INT,
 OUT p_name VARCHAR(50),
 OUT p_manager_id INT,
 OUT p_department VARCHAR(50),
 OUT p_salary INT
)
LANGUAGE plpgsql
AS $$
BEGIN
   SELECT emp_name, manager_id, department, salary
   INTO p_name, p_manager_id, p_department, p_salary
   FROM Employees
   WHERE emp_id = p_emp_id;
END;
$$;

CALL get_employee_by_id(3, NULL, NULL, NULL, NULL);
```

OUPUT:

	p_name character varying	p_manager_id integer	p_department character varying	p_salary integer
1	Suresh	1	Finance	85000

6. Learning Outcomes (What I have learned):

- **Stored Procedure Understanding:** I learned how to create and use stored procedures in PostgreSQL to perform database operations efficiently.
- **Practical Database Skills:** I understood how stored procedures can be used to manage employee data such as inserting, updating, deleting, and retrieving records in a relational database.
- **CRUD Operations Knowledge:** I learned how to implement Insert, Update, Delete, and Retrieve operations using stored procedures on the Employees table**.**
- **Parameter Usage:** I understood the use of different types of parameters such as IN, OUT, and INOUT in stored procedures for passing and returning values.
- **PL/pgSQL Practice:** I gained hands-on experience in writing procedural SQL code using PL/pgSQL syntax including BEGIN, END, and SQL statements inside procedures.
- **PostgreSQL / pgAdmin Practice:** I improved my practical skills in creating, executing, and testing stored procedures in PostgreSQL / pgAdmin.